

MySQL UPDATE utasítás

Mi az UPDATE utasítás?

Az UPDATE utasítással meglévő rekordok adatait tudjuk módosítani egy táblában. Ez az egyik legfontosabb DML (Data Manipulation Language) utasítás, amellyel az adatbázisban tárolt információkat frissíthetjük.

Alapszintaxis

```
UPDATE tabla_nev
SET oszlop1 = uj_ertek1, oszlop2 = uj_ertek2, ...
WHERE feltetel;
```

Szintaxis elemei:

- **UPDATE tabla_nev** - megadjuk, melyik táblát szeretnénk módosítani
- **SET** - kulcsszó, amely után felsoroljuk a módosítandó oszlopokat és az új értékeiket
- **WHERE** - feltétel, amely meghatározza, mely sorok legyenek módosítva
- ⚠ **FONTOS:** Ha elhagyjuk a WHERE feltételt, MINDEN sor módosul a táblában!

Példák egyszerű UPDATE utasításokra

1. Egy rekord egy mezőjének módosítása

```
-- Egy konkrét film évszámának módosítása
UPDATE filmek
SET ev = 1995
WHERE cim = 'Toy Story';
```

2. Egy rekord több mezőjének módosítása

```
-- Egy könyv árának és raktárkészletének frissítése
UPDATE konyvek
SET ar = 3500, keszlet = 15
WHERE isbn = '978-963-123-456-7';
```

3. Több rekord módosítása feltétel alapján

```
-- Az összes 2020 előtti film kategóriájának módosítása
UPDATE filmek
SET kategoria = 'klasszikus'
WHERE ev < 2020;
```

4. Számított értékkel történő módosítás

```
-- Minden termék árának 10%-os emelése
UPDATE termek
SET ar = ar * 1.10;

-- Csak a bizonyos kategória árának emelése
UPDATE termek
SET ar = ar * 1.15
WHERE categoria = 'elektronika';
```

5. NULL érték beállítása

```
-- Töröljük egy ügyfél telefonszámát (NULL-ra állítjuk)
UPDATE ugyfelek
SET telefon = NULL
WHERE id = 42;
```

6. Szöveges értékek módosítása

```
-- Email cím frissítése
UPDATE felhasznalok
SET email = 'uj.email@example.com'
WHERE felhasznalo_nev = 'janos123';

-- Szöveg összefűzése a meglévő értékkel
UPDATE konyvek
SET megjegyzes = CONCAT(megjegyzes, ' - Bestseller!')
WHERE eladott > 10000;
```

Speciális UPDATE esetek

1. Dátum értékek módosítása

```
-- Mai dátum beállítása
UPDATE rendelések
SET szállitasi_datum = CURDATE()
WHERE rendelés_id = 1001;

-- Dátum módosítása számítással
UPDATE események
SET datum = DATE_ADD(datum, INTERVAL 7 DAY)
WHERE helyszín = 'Budapest';
```

2. JOIN használata UPDATE-ben

```
-- Szerző nevének frissítése könyvek táblában egy másik tábla alapján
UPDATE könyvek k
INNER JOIN szerzok sz ON k.szerzo_id = sz.id
SET k.szerzo_nev = sz.teljes_nev
WHERE sz.becenév IS NOT NULL;
```

Biztonságos UPDATE gyakorlat

1. Mindig először nézd meg, mit fogsz módosítani!

```
-- ELŐBB futtasd le SELECT-tel:
SELECT * FROM filmek WHERE év < 2000;

-- HA rendben van, CSAK AKKOR módosítsd:
UPDATE filmek
SET kategoria = 'klasszikus'
WHERE év < 2000;
```

2. LIMIT használata teszteléskor

```
-- Csak 10 rekordot módosítunk tesztként
UPDATE termek
SET ar = ar * 1.05
WHERE kategoria = 'ruházat'
LIMIT 10;
```

Gyakori hibák és megoldások

✗ Hiba: WHERE feltétel nélküli UPDATE

```
-- VESZÉLYES! Minden sor módosul!
UPDATE diákok
SET osztály = '9.A';
```

Megoldás: Mindig használj WHERE feltételt (kivéve, ha tényleg minden sort módosítani akarsz)!

✗ Hiba: Safe Update Mode

MySQL Workbench-ben alapértelmezetten be van kapcsolva a "Safe Update" mód, amely megakadályozza a WHERE nélküli vagy kulcs nélküli UPDATE-et.

Hibaüzenet:

```
Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE that uses a KEY column.
```

Megoldás 1: Kapcsold ki ideiglenesen:

```
SET SQL_SAFE_UPDATES = 0;
-- UPDATE utasítás
SET SQL_SAFE_UPDATES = 1;
```

Megoldás 2: Használj kulcs oszlopot a WHERE feltételben:

```
UPDATE diákok
SET osztály = '9.A'
WHERE id = 5; -- id az elsődleges kulcs
```

✗ Hiba: Adattípus inkompatibilitás

```
-- Rossz: szöveget próbálunk számként megadni idézőjelek nélkül
UPDATE termek
SET nev = Laptop; -- Hiányzik az idézőjel!
```

Megoldás:

```
UPDATE termek
SET nev = 'Laptop'
WHERE id = 1;
```

UPDATE és az adatintegritás

Külső kulcsok kezelése

Ha egy tábla külső kulcsot tartalmaz, az UPDATE során ügyelni kell arra, hogy az új érték létezzen a hivatkozott táblában.

```
-- Ez sikertelen lesz, ha nem létezik categoria_id = 999
UPDATE termek
SET categoria_id = 999
WHERE id = 1;
-- Error: Cannot add or update a child row: a foreign key constraint fails
```

Megoldás: Először ellenőrizd, hogy létezik-e az új érték:

```
SELECT * FROM kategoriak WHERE id = 999;
```

Teljesítmény és optimalizálás

Indexek használata

Az UPDATE gyorsabb, ha indexelt oszlopon van a WHERE feltétel:

```
-- Gyors, mert 'id' indexelt (PRIMARY KEY)
UPDATE ugyfelek
SET email = 'uj@email.com'
WHERE id = 100;

-- Lassabb lehet, ha 'nev' nincs indexelve
UPDATE ugyfelek
SET email = 'uj@email.com'
WHERE nev = 'Nagy János';
```

Gyakorlati feladatok

1. feladat

Módosítsd az összes 5000 Ft-nál drágább könyv árát 10%-kal csökkentve!

► Megoldás

```
UPDATE konyvek
SET ar = ar * 0.90
WHERE ar > 5000;
```

2. feladat

Frissítsd az összes "aktív" státuszú rendelés állapotát "feldolgozás alatt"-ra, amelyek 2024-ben érkeztek!

► Megoldás

```
UPDATE rendelések
SET statusz = 'feldolgozás alatt'
WHERE statusz = 'aktív'
AND YEAR(rendeles_datum) = 2024;
```

Összefoglalás

Az UPDATE utasítás alapvető parancs az adatbázis-kezelésben:

- ☒ **Használd amikor:** módosítani kell meglévő adatokat
- ☒ **Mindig használd WHERE feltételt** (kivéve ha tudatosan minden sort akarsz módosítani)
- ☒ **Először SELECT-tel ellenőrizz**, aztán UPDATE-tel módosíts
- ☒ **Figyelj az adattípusokra** és külső kulcsokra
- ☒ **Tesztelj kis mintán** (LIMIT), mielőtt nagy mennyiségű adatot módosítanál

Emlékeztető: Az UPDATE visszavonhatatlan (tranzakción kívül), ezért mindig készíts mentést fontos adatok módosítása előtt!